Домашнее задание 11 — Fork/Join Framework

- **Дедлайн:** 28.10.2026
- **Тема:** Fork/Join Framework
- **Форма проверки:** задание с самопроверкой
- **Вариант итогового ДЗ:** 3 — парсер вакансий
- **Модуль:** `hw11.forkjoin` в Maven-проекте `app/`

---

1. Что требовалось и что сделано

Текст задания:

1. Реализовать подсчёт количества агрегированных элементов из прошлых домашних заданий.
2. Написать рекурсивную задачу через Fork/Join.
3. Разбить задачу на батчи: каждая пачка — новая задача.
4. Отчёт с этапами.

Сделано:

Что	Где
Агрегированная запись (результат парсера)	`model.AggregatedVacancy`
Критерий подсчёта (город, зарплата, слово в заголовке)	`model.CountCriteria`
Агрегат подсчёта и его сложение	`model.CountResult`
Детерминированный набор данных	`data.VacancyDataset`
Рекурсивная задача с разбиением по партиям	`count.BatchCountTask`
Статистика выполнения (партии, задачи, потоки)	`count.CounterStats`
Собственный `ForkJoinPool`	`count.ForkJoinCounter`
Последовательная база сравнения	`count.SequentialCounter`
Итог замера	`count.CountSummary`
Строки отчёта	`report.ForkJoinReport`
Демонстрация	`ForkJoinDemo`, запуск `--app.hw=hw11`
Настройки `app.hw11.*`	`config.Hw11Properties`

Набор данных — 1 000 000 агрегированных вакансий с зерном `2026`: это то, что парсер (ДЗ 8–10) уже собрал и сохранил. Считается число вакансий, подходящих под критерий «Томск, зарплата от 100 000 ₽, слово `java` в нормализованном заголовке», плюс суммарные величины (удалённые, сумма и максимум зарплат).

2. Этапы выполнения

Этап 1. Набор данных

Генератор с зерном выдаёт одинаковый набор при одинаковом зерне, поэтому замеры и ответ счётчика воспроизводимы. Заголовки и города берутся из фиксированных массивов, зарплата — 40 000…250 000 ₽, 15 % вакансий удалённые. Фактически: **1 000 000 записей за 400.5 мс**.

Почему набор большой: на десятках и тысячах записей рекурсивное разбиение только добавляет накладные расходы, и сравнивать параллельный подсчёт с последовательным бессмысленно. Проверка соответствия критерию включает нормализацию заголовка (нижний регистр, удаление пунктуации) — это та же операция, что делал конвейер ДЗ 10; без неё миллион записей считается быстрее, чем создаются задачи.

Этап 2. Последовательный подсчёт — база сравнения

Один проход по списку в текущем потоке:

```
Этап 2. Последовательный подсчёт — база сравнения
    последовательно: 147.857 мс, партий 1 из 1, задач 1, краж 0, потоков 1
    просмотрено 1000000, подошло 18067, из них удалённых 2676, сумма зарплат 3167669461 ₽, сре
    поток: time-boxed-task
```

Подошло **18 067** вакансий из миллиона. Это число — эталон: любой параллельный вариант обязан дать тот же `CountResult` (record сравнивается по значению).

Этап 3. Рекурсивная задача и разбиение на партии

`BatchCountTask` — `RecursiveTask<CountResult>` с диапазоном `[from, to)`, критерием и размером партии.

- Если диапазон не больше партии — это **лист**: он считает свои записи последовательно, а `CounterStats` фиксирует партию.
- Иначе диапазон делится на две части **по границе партий**: `mid = from + (партий слева) * batchSize`. Левая половина уходит в `fork()`, правая считается в текущем потоке (`compute()`), затем результаты складываются `join()`-ом. Так дерево задач остаётся сбалансированным, а листьев ровно `ceil(size / batchSize)`.

Почему не «делить пополам по индексу»: при `size = 1000`, `batchSize = 300` деление пополам даёт листья 250/250/500, и обещание «каждая пачка — новая задача» перестаёт быть проверяемым. При выравнивании по партиям для тех же чисел листья — 300/300/300/100, а их число равно `ceil(1000/300) = 4`. Тест проверяет это равенство для разных размеров и партий.

Этап 4. Замеры: размер партии

Пул на 8 потоков, каждый замер повторён дважды (в таблицу идёт лучшее время — одиночный замер на JVM шумит):

```
Этап 3. Fork/Join: зависимость от размера партии (параллелизм 8)
    вариант                              время, мс  ускорение   партий    задач     краж потокс
    Fork/Join (партия 1000)                 21.245     6.96×     1000     1999       22       8
    Fork/Join (партия 5000)                 24.112     6.13×      200      399       22       8
    Fork/Join (партия 20000)                23.024     6.42×       50       99       13       8
    Fork/Join (партия 100000)               17.868     8.27×       10       19       10       8
```

Число задач = `2 × партий − 1`: внутренние узлы дерева тоже задачи. Для 200 партий это 399 задач — сходится с теорией полного двоичного дерева. «Кражи» — это `ForkJoinPool.getStealCount()` за прогон (разница до и после), то есть признак того, что простаивающие потоки реально забирали работу у занятых.

Этап 5. Замеры: параллелизм

```
Этап 4. Fork/Join: зависимость от параллелизма (партия 5000)
    Fork/Join (партия 5000)                25.467     5.81×      200      399        3       2
        потоки: [hw11-forkjoin-1, hw11-forkjoin-2]
    Fork/Join (партия 5000)                21.288     6.95×      200      399        9       4
        потоки: [hw11-forkjoin-1, hw11-forkjoin-2, hw11-forkjoin-3, hw11-forkjoin-4]
    Fork/Join (партия 5000)                11.808    12.52×      200      399       21       8
        потоки: [hw11-forkjoin-1, ..., hw11-forkjoin-8]
```

Время падает с ростом параллелизма, а число задействованных потоков совпадает с размером пула. На 8 потоках ускорение 12.52× относительно последовательного варианта — больше 8 из-за того, что последовательный проход делает ещё и нормализацию заголовков в одном потоке, а также из-за прогрева JIT (база измерена раньше и «холоднее»). Это нормальный для учебного замера эффект, поэтому в отчёте приведены и абсолютные времена, и число потоков, а не только коэффициент.

3. Фактический вывод демонстрации

```
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw11 --server.port=0"
```

Прогон завершился с кодом `0`, живых потоков ДЗ 11 после завершения нет.

```
=== ДЗ 11. Fork/Join: рекурсивный подсчёт агрегированных вакансий партиями ===


Процесс: pid 128693
Настройки ДЗ 11:
    - размер набора: 1000000 записей, зерно 2026
    - критерий: город: Томск, зарплата от 100000 ₽, слово в заголовке: java
    - размер партии по умолчанию: 5000, параллелизм: 8
    - повторов на замер: 2 (в таблицу попадает лучшее время)

Этап 1. Подготовка набора агрегированных записей
    сгенерировано 1000000 записей за 400.524 мс, городов 10, заголовков 12

Этап 2. Последовательный подсчёт — база сравнения
    последовательно: 147.857 мс, партий 1 из 1, задач 1, краж 0, потоков 1
    просмотрено 1000000, подошло 18067, из них удалённых 2676, сумма зарплат 3167669461 ₽, сре
    партии соответствуют ожидаемым: да (1 из 1)
    поток: time-boxed-task

Этап 3. Fork/Join: зависимость от размера партии (параллелизм 8)
```

```
    вариант                              время, мс  ускорение   партий    задач     краж потокс
    Fork/Join (партия 1000)                 21.245     6.96×     1000     1999       22       8
    Fork/Join (партия 5000)                 24.112     6.13×      200      399       22       8
    Fork/Join (партия 20000)                23.024     6.42×       50       99       13       8
    Fork/Join (партия 100000)               17.868     8.27×       10       19       10       8

    последовательно 147.857 мс, Fork/Join 17.868 мс — ускорение 8.27× при параллелизме 8
    агрегат совпадает с последовательным: да
    Fork/Join (партия 100000): 17.868 мс, партий 10 из 10, задач 19, краж 10, потоков 8

Этап 4. Fork/Join: зависимость от параллелизма (партия 5000)
    вариант                              время, мс  ускорение   партий    задач     краж потокс
    Fork/Join (партия 5000)                 25.467     5.81×      200      399        3       2
        потоки: [hw11-forkjoin-1, hw11-forkjoin-2]
    Fork/Join (партия 5000)                 21.288     6.95×      200      399        9       4
        потоки: [hw11-forkjoin-1, hw11-forkjoin-2, hw11-forkjoin-3, hw11-forkjoin-4]
    Fork/Join (партия 5000)                 11.808    12.52×      200      399       21       8
        потоки: [hw11-forkjoin-1, hw11-forkjoin-2, hw11-forkjoin-3, hw11-forkjoin-4, hw11-fork
    последовательно 147.857 мс, Fork/Join 11.808 мс — ускорение 12.52× при параллелизме 8

Этап 5. Проверка корректности разбиения
    Fork/Join (партия 5000): 11.878 мс, партий 200 из 200, задач 399, краж 22, потоков 8
    просмотрено 1000000, подошло 18067, из них удалённых 2676, сумма зарплат 3167669461 ₽, сре
    партии соответствуют ожидаемым: да (200 из 200)
    партий ровно ceil(1000000 / 5000) = 200: да
    агрегат равен последовательному: да
    задач больше, чем партий, потому что внутренние узлы тоже задачи: 399 против 200
    украдено задач пулом за прогон: 22 (кража — признак реального параллелизма, а не работы од

[tpm] Демонстрация завершена: параллельный подсчёт совпал с последовательным, пул остановлен

живых потоков ДЗ 11 после завершения: []
```

4. Промежуточные результаты по этапам

Этап	Фактический результат
Набор	1 000 000 записей за 400.5 мс, зерно 2026
Последовательный подсчёт	**147.857 мс**, подошло 18 067, сумма 3 167 669 461 ₽, максимум 249 999 ₽
Партия 1000	21.245 мс, 1000 партий, 1999 задач, 22 кражи, 8 потоков
Партия 5000	24.112 мс, 200 партий, 399 задач
Партия 20 000	23.024 мс, 50 партий, 99 задач
Партия 100 000	17.868 мс, 10 партий, 19 задач
Параллелизм 2 / 4 / 8	25.467 / 21.288 / 11.808 мс (ускорение 5.81× / 6.95× / 12.52×)

Этап	Фактический результат
Корректность	агрегат совпал с последовательным во всех прогонах; партий ровно `ceil(size/batch)`

5. Почему выравнивание по партиям и почему пул свой

- **Проверяемость разбиения.** При делении «пополам по индексу» листья получаются произвольного размера, и нельзя утверждать ни «каждая пачка — новая задача фиксированного размера», ни точное число партий. Выравнивание по границе партии делает число листьев предсказуемым (`ceil(size / batchSize)`) и проверяемым тестом.
- **Нет общего состояния.** Задача хранит только `final`-поля и общий атомарный счётчик статистики, поэтому «украденная» подзадача корректно считается тем потоком, который её выполнил.
- **Ассоциативное сложение.** `CountResult.plus` ассоциативен и коммутативен, поэтому порядок завершения подзадач на ответ не влияет — это обязательное условие корректности Fork/Join.
- **Свой пул.** `ForkJoinPool.commonPool()` общий для всего приложения: на его числа влияет чужая нагрузка, а размер не является параметром эксперимента. Здесь пул создаётся с явным параллелизмом и именованными потоками `hw11-forkjoin-N` и закрывается по завершении.
- **Демон-потоки** пула не мешают процессу завершиться даже при аварии, но пул всё равно закрывается явно (`close()` → `shutdown` → ожидание → при необходимости `shutdownNow`).

6. Автоматические проверки

```
mvn -pl app test -Dtest='ru.tsu.tpm.vacancyparser.hw11.**'
```

Результат: `Tests run: 35, Failures: 0, Errors: 0, Skipped: 0`.

Тест	Что проверяет
`BatchCountTaskTest` (5)	лист считает ровно свой диапазон; разбиение на `ceil(size/batch)` листьев и `2B−1` задач; пустой диапазон; сложение результатов детей; отказ некорректных диапазонов и партий
`ForkJoinCounterTest` (9)	совпадение с последовательным для размеров 0…1001 и партий 1…5000; число задач; именованные потоки пула и предел параллелизма; пустой набор; `close()`; некорректные параметры; критерий действительно сужает выборку
`CountCriteriaTest` (4)	город/зарплата/слово, нормализация регистра и пунктуации, пустые условия, отрицательная граница
`CountResultTest` (5)	нейтральный элемент, ассоциативность сложения, средняя и максимум, строка отчёта, валидация записи
`VacancyDatasetTest` (3)	детерминированность по зерну, диапазоны значений, пустой и отрицательный размер

Тест	Что проверяет
`ForkJoinReportTest` (4)	ускорение и строка сравнения, колонки таблицы, проверка числа партий, сравнение агрегатов
`Hw11PropertiesBindingTest` (2)	значения по умолчанию и переопределение `app.hw11.*`, сборка критерия
`ForkJoinDemoConditionTest` (3)	демо только при `--app.hw=hw11`

Сборка целиком: `mvn -pl app package` — успешно. Полный прогон всех домашних заданий: `Tests run: 545, Failures: 0, Errors: 0, Skipped: 0`.

7. Соответствие чек-листу

Пункт задания	Где подтверждено	Отметка
Подсчёт количества агрегированных элементов из прошлых ДЗ	`AggregatedVacancy` + `CountCriteria`	✅
Рекурсивная задача через Fork/Join	`BatchCountTask extends RecursiveTask`	✅
Разбиение на батчи, каждая пачка — новая задача	`mid = from + (партий слева) * batchSize`; тест числа партий	✅
Отчёт с этапами	Этот файл, разделы 2–4	✅
Корректность результата	сравнение с последовательным во всех прогонах и тестах	✅

8. Как запустить и сдать

```
cd "Теория и практика многопоточности"

# Полная демонстрация (завершается с кодом 0)
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw11 --server.port=0"

# Другой размер набора и партии
mvn -pl app spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw11 --app.hw11.dataset-size

# Готовый jar
mvn -pl app package
java -jar app/target/vacancy-parser.jar --app.hw=hw11 --server.port=0
```

Параметры: `app.hw11.dataset-size`, `app.hw11.batch-size`, `app.hw11.pool-size`, `app.hw11.batch-sizes`, `app.hw11.pool-sizes`, `app.hw11.repeats`, `app.hw11.seed`, `app.hw11.criteria-city`, `app.hw11.criteria-min-salary`, `app.hw11.criteria-keyword`.

Сдача: скопировать отчёт в Google/Яндекс Документ, приложить исходники или ссылку на репозиторий, открыть доступ и прикрепить ссылку в личном кабинете.

9. Примечание о числах и окружении

Все числа — фактические замеры на машине с 12 ядрами (OpenJDK 21.0.13). В таблицы попадает лучшее из двух прогонов, потому что одиночный замер на JVM шумит: первый прогон интерпретируется, затем компилируется C1/C2. Абсолютные времена зависят от машины, структура зависимостей (партия, параллелизм, число краж, число задач) устойчива.